MLP Coursework – Deep Learning Module

Problem 2 – Solution

1. Is this ANN suitable for multiclass classification?

No, not as it is. The network has a single output node with a sigmoid activation, so it gives one number $\hat{y}$ between 0 and 1. That can be read as P(class = 1), and P(class = 0) = $1 - \hat{y}$, so it can only separate **two** classes (binary classification).

For K classes (K > 2) we have a few problems with this setup:

- One output cannot hold K different labels. If we cut the 0–1 range into regions (e.g. <0.33 is class A, 0.33–0.66 is class B...) we force an order on the classes that does not really exist, and the loss treats class A and C as 'further apart' than A and B.
- Independent sigmoid outputs do not give a proper probability distribution over classes (they need not sum to 1).

Fix: use K output nodes (one per class), with a softmax activation in the output layer and one-hot encoded labels, trained with categorical cross-entropy. The predicted class is the node with the highest probability.

2. Change in formula 1.4 for 2 output nodes

Now there are two output nodes OP1 and OP2, with outputs $\hat{y}_1$, $\hat{y}_2$ and ground truths y_1, y_2 (one-hot, e.g. class 1 → (1, 0)). Each output node has its own weights $W_{i,op1}$, $W_{i,op2}$ and biases $W_{0,op1}$, $W_{0,op2}$. So the parameters become $12 + 2 \times 4 = 20$.

Forward pass for output k (k = 1, 2):

$$z_{opk} = W_{0,opk} + W_{1,opk}h_1 + W_{2,opk}h_2 + W_{3,opk}h_3, \quad \hat{y}_k = \sigma(z_{opk})$$

MSE now sums over both outputs:

$$L = \tfrac{1}{2}[(y_1 - \hat{y}_1)^2 + (y_2 - \hat{y}_2)^2]$$

Output weights: each output node is handled like before, only with its own error term.

$$\delta_k = (\hat{y}_k - y_k) \cdot \hat{y}_k(1 - \hat{y}_k)$$

$$\boldsymbol{\partial L/\partial W_{i,opk} = \delta_k \cdot h_i, \quad \partial L/\partial W_{0,opk} = \delta_k}$$

H3 weights: this is the part that changes. H3 is connected to *both* output nodes, so H3 affects both errors and the gradients from the two paths get added:

$$\partial L/\partial h_3 = \delta_1 \cdot W_{3,op1} + \delta_2 \cdot W_{3,op2}$$

$$\delta_3 = (\delta_1 W_{3,op1} + \delta_2 W_{3,op2}) \cdot h_3(1 - h_3)$$

$$\boldsymbol{\partial L/\partial W_{i3} = \delta_3 \cdot x_i, \quad \partial L/\partial W_{03} = \delta_3}$$

(Earlier, with only one output, δ_3 had just one term, $\delta_{op} \cdot W_{3op}$.)

3. Change in formula 1.4 for binary cross-entropy (BCE) loss

Back to a single output node. Loss for one sample:

$$L = -[\, y \ln(\hat{y}) + (1 - y) \ln(1 - \hat{y}) \,]$$

Derivative w.r.t. $\hat{y}$:

$$\partial L/\partial \hat{y} = -y/\hat{y} + (1 - y)/(1 - \hat{y})$$

$$= [\, -y(1 - \hat{y}) + (1 - y)\hat{y} \,] / [\, \hat{y}(1 - \hat{y}) \,]$$

$$= (\hat{y} - y) / [\, \hat{y}(1 - \hat{y}) \,]$$

Now multiply by the sigmoid derivative $\partial \hat{y}/\partial z_{op} = \hat{y}(1 - \hat{y})$:

$\delta_{op} = \partial L/\partial z_{op} = (\hat{y} - y) / [\ \hat{y}(1 - \hat{y})\ ] \times \hat{y}(1 - \hat{y}) = \mathbf{\hat{y} - y}$

The $\hat{y}(1 - \hat{y})$ term cancels out. So only δ_{op} changes, and everything else in 1.4 follows from it:

$$\mathbf{\partial L/\partial W_{iop} = (\hat{y} - y) \cdot h_i,} \quad \mathbf{\partial L/\partial W_{0op} = (\hat{y} - y)}$$

$$\delta_3 = (\hat{y} - y) \cdot W_{3op} \cdot h_3(1 - h_3)$$

$$\mathbf{\partial L/\partial W_{i3} = (\hat{y} - y) \cdot W_{3op} \cdot h_3(1 - h_3) \cdot x_i,} \quad \mathbf{\partial L/\partial W_{03} = \delta_3}$$

Compared with MSE, the factor $\hat{y}(1 - \hat{y})$ is gone from the gradients. With MSE this factor becomes very small when the output saturates ($\hat{y}$ close to 0 or 1), even if the prediction is wrong, so learning is slow. With BCE the gradient stays proportional to the error $(\hat{y} - y)$, so training is faster and more stable. That is why BCE is preferred for classification.